

Student ID:		24CS072	Student Name:		Patel Prince
Subject code & Name	:	CSE304 & Full Stack Development	Practical	:	5 Academic Year : 2025-26

Task - 5

Aim

- Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose.
- Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now).
- Replace the in-memory array from Practical 4 with real Mongoose model operations.
- Test all CRUD operations against the live database using Postman.
- Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.

Code

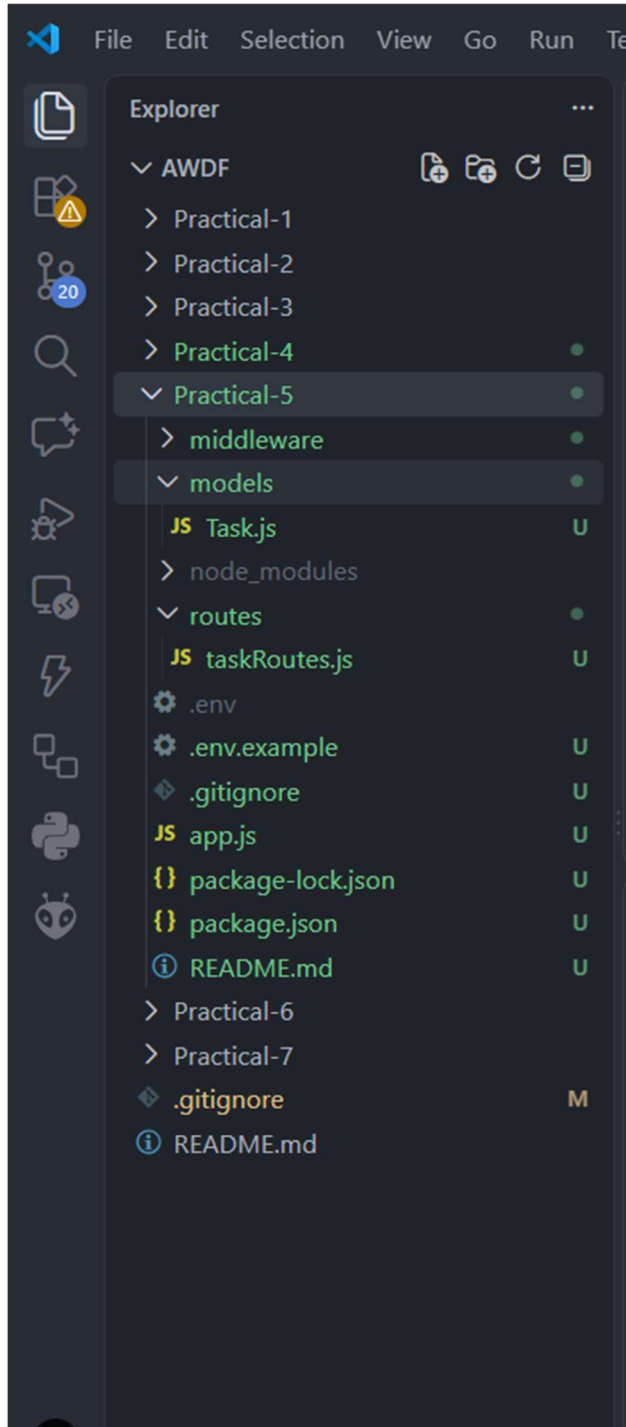
GitHub Link Repository Link:

Code repo :-

<https://github.com/PRINCE-24CS072/AWDF/tree/main/Practical-5>

Output

Folder Structure



MONGODB COMPASS

The image displays two screenshots. The top screenshot shows the MongoDB Compass interface. The 'tasks' collection is selected under the 'test' database. The 'Indexes' tab is active, showing a single index on the '_id' field. A notification banner indicates that search indexes can be created and viewed under 'Search and Vector Search'.

Name & Definition	Type	Size	Usage	Properties	Status
id	REGULAR	4.1 kB	1 (since Sat Sep 05 2026)	UNIQUE	READY

The bottom screenshot shows a web browser with a REST client open to the URL 'http://localhost:3000/tasks'. The 'GET' method is selected, and the 'Send' button is visible. The 'Body' tab is active, showing a status of '200 OK' with a response time of 634 ms and a body size of 235 B.

The image displays two screenshots of a REST client interface, likely Postman, showing a successful POST and GET request to a local API.

Top Screenshot (POST Request):

- URL:** `http://localhost:3000/tasks`
- Method:** `POST`
- Body (raw):**

```
1 {
2   "title": "Learn MongoDB",
3   "description": "Complete Practical 5",
4   "completed": false,
5   "priority": "high"
6 }
```
- Response (JSON):**

```
1 {
2   "message": "Task created successfully",
3   "task": {
4     "title": "Learn MongoDB",
5     "description": "Complete Practical 5",
6     "completed": false,
7     "priority": "high",
8     "_id": "6a9c6078cf85a7cd41ba5911",
9     "createdAt": "2026-09-05T18:33:28.196Z",
10    "__v": 0
11  }
12 }
```
- Status:** 201 Created, 861 ms, 466 B

Bottom Screenshot (GET Request):

- URL:** `http://localhost:3000/tasks`
- Method:** `GET`
- Query Params:** (Empty table)
- Response (JSON):**

```
1 [
2   {
3     "_id": "6a9c6078cf85a7cd41ba5911",
4     "title": "Learn MongoDB",
5     "description": "Complete Practical 5",
6     "completed": false,
7     "priority": "high",
8     "createdAt": "2026-09-05T18:33:28.196Z",
9     "__v": 0
10   }
11 ]
```
- Status:** 200 OK, 39 ms, 416 B

The image displays two screenshots of a REST client interface, likely Postman, showing a sequence of API calls to a task management service.

Top Screenshot (GET Request):

- Method:** GET
- URL:** `http://localhost:3000/tasks/6a9c6078cf85a7cd41ba5911`
- Status:** 200 OK
- Response Time:** 45 ms
- Response Size:** 414 B
- Body (JSON):**

```
{  1  {  2    "_id": "6a9c6078cf85a7cd41ba5911",  3    "title": "Learn MongoDB",  4    "description": "Complete Practical 5",  5    "completed": false,  6    "priority": "high",  7    "createdAt": "2026-09-05T18:33:28.196Z",  8    "__v": 0  9  }
```

Bottom Screenshot (PUT Request):

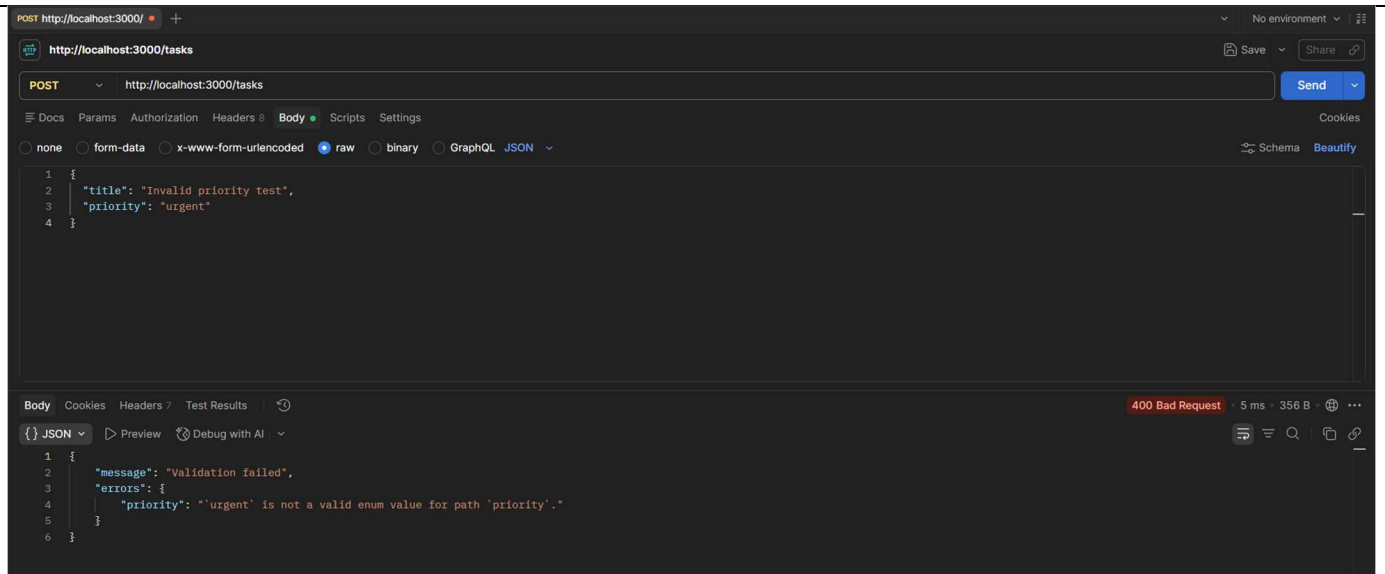
- Method:** PUT
- URL:** `http://localhost:3000/tasks/6a9c6078cf85a7cd41ba5911`
- Status:** 200 OK
- Response Time:** 50 ms
- Response Size:** 461 B
- Body (JSON):**

```
{  1  {  2    "message": "Task updated successfully",  3    "task": {  4      "_id": "6a9c6078cf85a7cd41ba5911",  5      "title": "Learn MongoDB",  6      "description": "Complete Practical 5",  7      "completed": false,  8      "priority": "high",  9      "createdAt": "2026-09-05T18:33:28.196Z", 10     "__v": 0 11   } 12 }
```

The image displays two screenshots of a REST client interface, likely Postman, showing HTTP requests and responses.

Top Screenshot: A DELETE request to `http://localhost:3000/tasks/6a9c6078cf85a7cd41ba5911`. The response is a 200 OK status with a body containing `"message": "Task deleted successfully"`.

Bottom Screenshot: A POST request to `http://localhost:3000/tasks` with a body containing `{ "description": "No title" }`. The response is a 400 Bad Request status with a body containing `{ "message": "Validation failed", "errors": { "title": "Title is required" } }`.



Terminal

```
PS C:\Users\Prince Patel\OneDrive\Desktop\LAB Submission\AWDF\Practical-5> npm start
Server running at http://localhost:3000
GET / - 5/9/2026, 11:54:38 pm
GET /tasks - 5/9/2026, 11:55:07 pm
GET /tasks/:id - 5/9/2026, 11:55:11 pm
GET /tasks/:id - 5/9/2026, 11:55:14 pm
GET /tasks/:id - 5/9/2026, 11:55:16 pm
GET /tasks - 5/9/2026, 11:55:37 pm
GET /tasks - 6/9/2026, 12:02:25 am
POST /tasks - 6/9/2026, 12:03:28 am
GET /tasks - 6/9/2026, 12:16:35 am
GET /tasks/6a9c6078cf85a7cd41ba5911 - 6/9/2026, 12:18:30 am
PUT /tasks/YOUR_ID - 6/9/2026, 12:19:27 am
PUT /tasks/6a9c6078cf85a7cd41ba5911 - 6/9/2026, 12:19:40 am
DELETE /tasks/6a9c6078cf85a7cd41ba5911 - 6/9/2026, 12:20:30 am
```